Reporte

<table>
<tr><td colspan="3">Equipo 16</td></tr>
<tr><td colspan="2">Nombre:

Lesly Adamaris Cotlame Nieto

Christopher Bethancourt Contreras

Francisco Mora Ruiz</td><td>Matrícula:

2226958

2226948

2226912</td></tr>
<tr><td>Nombre del curso: Componentes de lógica de negocios.</td><td colspan="2">Nombre del profesor: Miguel Ángel Gómez Marroquín</td></tr>
<tr><td>Módulo Netacad: 3 — PIA, apartado 2: Arquitectura</td><td colspan="2">Actividad: Patrones de diseño y principios SOLID aplicados en Eventos Nova.</td></tr>
<tr><td colspan="3">Fecha: 05/11/2026 (revisión documental) · Entrega oficial del PIA: 06/11/2026</td></tr>
<tr><td colspan="3">Bibliografía (Formato APA)

• Evans, E. (2003). Domain-driven design: Tackling complexity in the heart of software. Addison-Wesley.
• Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.
• Martin, R. C. (2017). Clean architecture: A craftsman’s guide to software structure and design. Prentice Hall.
• PostgreSQL Global Development Group. (s. f.). PostgreSQL 18 documentation. https://www.postgresql.org/docs/18/
• SQLAlchemy. (s. f.). SQLAlchemy 2.0 documentation. https://docs.sqlalchemy.org/en/20/</td></tr>
</table>

Objetivo:

Documentar la arquitectura implementada de Eventos Nova y justificar sus decisiones con evidencia del código, de las migraciones y de las pruebas, de modo que exista trazabilidad entre el análisis del negocio, el diseño y la implementación.

Procedimiento:

Patrones de diseño y principios SOLID - Eventos Nova

Introducción

Este documento identifica los patrones de diseño que están implementados en el código de Eventos Nova, indica su ubicación y su finalidad, y explica los cinco principios SOLID con ejemplos del propio sistema. Solo se incluye un patrón cuando existe una clase o función concreta en `src/` que lo implementa; los patrones mencionados en documentos anteriores pero sin implementación se listan aparte.

Arquitectura en la que se ubican los patrones

El sistema sigue una arquitectura en capas con Domain Model (ADR-001) y puertos y adaptadores. La API (`src/api/`) recibe las peticiones y aplica la seguridad; la capa de aplicación (`src/application/`) orquesta los casos de uso; el dominio (`src/domain/`) contiene los agregados y sus reglas; y la infraestructura (`src/infrastructure/`) implementa la persistencia en PostgreSQL, el Outbox y los adaptadores.

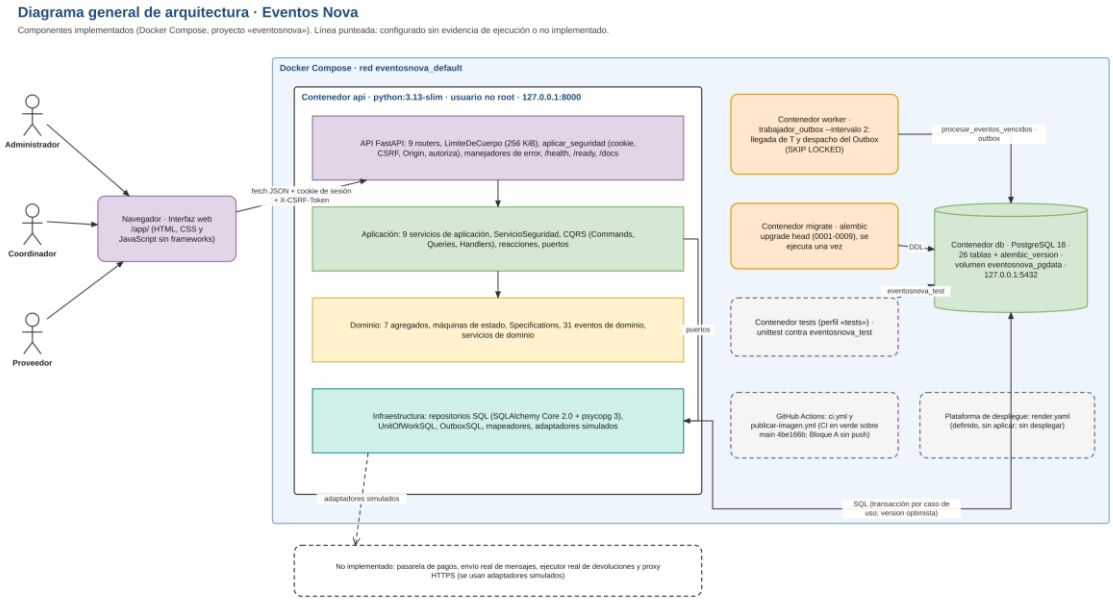


Figura 1. Diagrama general de arquitectura (fuente editable en 02_ARQUITECTURA/03_ARQUITECTURA_SISTEMA).

Patrones implementados

Patrón	Finalidad en Eventos Nova	Ubicación	Ejemplo real
Domain Model y agregados (ADR-001)	Ubicar reglas de pagos, plazos y aprobaciones en el concepto responsable	src/domain/base.py (AggregateRoot); 7 raíces en evento.py, propuesta.py, contrato.py, estado_cuenta.py, solicitudes.py	MAQUINA_EVENTO solo permite pasar de «En organización» a «En ejecución» con la acción INICIAR
Value Object	Representar importes, datos y condiciones inmutables	src/domain/valores.py (28 dataclass inmutables: Importe, PlazoPago, CondicionesContractuales...)	Importe usa Decimal y rechaza negativos
State Machine	Transiciones explícitas y verificables	src/domain/maquina_estados.py (Transicion, MaquinaEstados) y máquinas de cada agregado	MAQUINA_EVENTO.transitar() lanza TransicionNoPermitida
Specification	Reglas combinables con motivo de rechazo	src/domain/especificacion.py (Y, O, No, ConMotivo) y especificaciones.py	AnticipacionMinima(5) para servicios adicionales; SinSaldoVencido
Domain Events	Desacoplar reacciones entre agregados	src/domain/eventos_dominio.py (31 eventos); AggregateRoot.extraer_eventos()	ContratoFirmado → ReaccionesPagos.al_firmarse_contrato → registrar_obligacion_anticipo
Repository	Aislar la persistencia del dominio	src/application/puertos.py (8 interfaces); repositorios_memoria.py y persistencia/repositorios_sql.py	EventoRepositorySQL implementa IEventoRepository
Data Mapper	Traducir agregados a filas sin ORM	persistencia/mapeadores.py (Mapeador y 7 mapeadores)	MapeadorEvento rehidrata Evento con sus servicios contratados
Unit of Work	Una transacción por caso de uso	IUnitOfWork; UnitOfWorkMemoria; UnitOfWorkSQL; servicios.py transaccion()	with transaccion(self.uow, ec): ... guardar(ec)
Transactional Outbox (ADR-004, ADR-005)	Entregar eventos de dominio sin pérdida	persistencia/outbox_sql.py (OutboxSQL, DespachadorOutboxSQL); trabajador_outbox.py	El trabajador reclama registros con FOR UPDATE SKIP LOCKED
Service Layer	Orquestar casos de uso con rutas delgadas	src/application/servicios.py (9 servicios) y seguridad.py (ServicioSeguridad)	Cada ruta de la API llama a un método de servicio
CQRS ligero	Separar lecturas de escrituras	src/application/cqrs.py (4 Commands, 11 Queries, 15 Handlers, 20 ViewModels)	GET /eventos/{evento_id}/pagos → ConsultarEstadoPagosHandler
Ports and Adapters	Integraciones intercambiables	Puertos IVerificadorPagos, IEjecutorDevoluciones, INotificadorParticipantes, IConsultaDisponibilidadProveedor; adaptadores.py	VerificadorPagosSimulado (integración simulada)
Optimistic Locking (ADR-006)	Evitar actualizaciones perdidas	Columna version (0004); ConflictoDeConcurrencia (persistencia/errores.py)	UPDATE condicionado por versión; respuesta 409
Composition Root	Ensamblar dependencias en un único lugar	infrastructure/composicion.py y persistencia/composicion_persistente.py	crear_app(fabrica_contenedor) en api/main.py

Patrón	Finalidad en Eventos Nova	Ubicación	Ejemplo real
Decorador de clase	Convertir ErrorDominio en rechazo de negocio	servicios.py con_rechazos_de_negocio	Los métodos devuelven Resultado.rechazo(...)
Middleware / inyección de dependencias de FastAPI	Seguridad y límites transversales	api/seguridad.py aplicar_seguridad; api/limites.py LimiteDeCuerpo	Respuestas 401, 403 y 413

Patrones exigidos por el PIA

El PIA solicita expresamente DDD, Specification y State Machine. Los tres están implementados y probados:

Elemento	Descripción
DDD	7 raíces de agregado que heredan de AggregateRoot, 7 entidades internas, 28 objetos de valor, 31 eventos de dominio, 2 servicios de dominio y 8 repositorios por agregado.
Specification	clase base Especificacion con los combinadores Y, O, No y ConMotivo (src/domain/especificacion.py) y 12 clases de especificación concretas en src/domain/especificaciones.py (ContratoEstaFirmado, PoliticaNoReembolsoComunicada, AnticipoVerificado, AfectaServicios, AfectaCostos, AfectaCondiciones, AnticipacionMinima, EventoNoIniciado, EventoNoCancelado, ElegibilidadObtenida, CondicionTemporalCumplida y SinSaldoVencido), combinadas en reglas nombradas como CONTRATACION_CONFIRMABLE, CAMBIO_CON_CARGO_TARDIO (menos de 7 días naturales), SERVICIO_ADICIONAL_EN_PLAZO (al menos 5 días naturales) y SOLICITUD_PUEDE_CONTINUAR; probadas en test_especificacion_combinadores.py, test_especificaciones_negocio.py y test_especificaciones_conservacion.py.
State Machine	MaquinaEstados genérica (src/domain/maquina_estados.py) y 9 máquinas declarativas: Evento, Contrato, ObligacionPago, Pago, Devolución, SolicitudCambio, Compatibilidad en evaluación (dentro de la solicitud de cambio), SolicitudServicioAdicional y SustitucionProveedor, probadas en los módulos test_*_maquina_estados.py.

Principios SOLID

S - Responsabilidad única

Cada clase tiene un único motivo de cambio. `Importe` solo modela dinero; `MaquinaEstados` solo resuelve transiciones; `AggregateRoot` solo registra eventos;

`LimiteDeCuerpo` solo limita el tamaño de la petición. Cada servicio de aplicación agrupa un proceso del negocio (propuesta, contratación, pagos, cierre del evento, cambios, adicionales, sustituciones, asignación y catálogo).

Limitación reconocida: `servicios.py` concentra 1,142 líneas con 9 clases y `GestionPagosService` expone 12 operaciones; es candidato a dividirse en una versión posterior.

O - Abierto/cerrado

Las reglas nuevas se agregan como nuevas `Especificacion` combinables con `Y`, `O` y `No` sin modificar las existentes; los estados y acciones nuevos son nuevas `Transicion` en la tabla de una `MaquinaEstados`; las reacciones nuevas son nuevas suscripciones en `reacciones.py` (por ejemplo, `s(E.EventoIniciado, ...)`).

L - Sustitución de Liskov

`EventoRepositoryMemoria` y `EventoRepositorySQL` implementan `IEventoRepository` y los servicios funcionan igual con ambos: la misma suite de pruebas se ejecuta en memoria y en PostgreSQL (clases de prueba como `DecisionesEnMemoria` y `DecisionesEnPostgreSQL`). Las especificaciones compuestas (`Y`, `O`, `No`, `ConMotivo`) se usan donde se espera una `Especificacion`.

I - Segregación de interfaces

Los puertos son pequeños y específicos: `IVerificadorPagos.verificar()`, `IEjecutorDevoluciones.ejecutar()`, `INotificadorParticipantes.notificar()` e `IConsultaDisponibilidadProveedor.consultar()` tienen un solo método; los repositorios son uno por agregado.

D - Inversión de dependencias

La capa de aplicación define los puertos y la infraestructura los implementa. Los servicios reciben repositorios, unidad de trabajo y adaptadores por constructor desde la raíz de composición. La dirección de dependencias se verificó por importaciones: `domain` no importa de otra capa y `application` solo importa de `domain`.

Patrones no implementados

Para evitar afirmaciones sin respaldo, se declara que no existen en el código: Event Sourcing (el estado se guarda en tablas), Saga u orquestador de procesos (la coordinación se realiza con eventos y reacciones), Factory o Strategy formales, Circuit Breaker o reintentos con espera progresiva, ni caché.

Conclusión

Los patrones se aplicaron para resolver problemas concretos del negocio: mantener una sola versión vigente de la propuesta, impedir confirmaciones sin anticipo verificado, separar las obligaciones por concepto y garantizar que las reacciones a los hechos de negocio no se pierdan. La cobertura de la medición archivada del 03/11/2026 (98.19 % de líneas y 93.20 % de ramas, con 1,654 pruebas aprobadas) respalda que estas estructuras están ejercitadas por pruebas. El Bloque A (05/11/2026) no modificó ningún patrón: sus cambios son de presentación y de los datos que devuelve la vista del proveedor.

Corrección del 05/11/2026

La versión del 03/11/2026 enumeraba 8 máquinas de estado y hablaba de «12 reglas de negocio». Verificado en el código del commit e3c98d9 (el commit 9c72bc7, publicado después, solo añade una regla de .gitattributes): existen 9 instancias de MaquinaEstados (falta añadir Compatibilidad en evaluación, definida en src/domain/solicitudes.py) y las 12 son clases de especificación concretas; las reglas nombradas son composiciones de esas clases. El estado técnico menciona «11 reglas nombradas»; en especificaciones.py se identificaron 8 composiciones a nivel de módulo (CONTRATACION_CONFIRMABLE, CAMBIO_REQUIERE_APROBACION_CLIENTE, MODIFICACION_CON_IMPACTO, CAMBIO_SIN_CARGO_TARDIO, CAMBIO_CON_CARGO_TARDIO, SOLICITUD_CAMBIO_EN_PLAZO, SERVICIO_ADICIONAL_EN_PLAZO y SOLICITUD_PUEDE_CONTINUAR). La diferencia se registra para revisión del equipo.